BLG 475E – Software Quality and Testing

Term Project Report:

LLM-Based Test Generation and Evaluation
Using the HumanEval Dataset

Ahmet Enes Çiğdem
150220079

Ali Eren Çiftçi
150220022

Taha Çali
150220050

April 27, 2026

Abstract

Contents

1 Introduction and Literature Review

1.1 Introduction

1.2 Literature Review

This section reviews five recent studies that investigate the use of LLMs in automated test generation and code quality assurance.

1.2.1 ChatUniTest: A Framework for LLM-Based Test Generation

Chen et al. [1] present *ChatUniTest*, an automated unit test generation framework that addresses two key weaknesses of LLM-based tools: limited context windows and insufficient validation of generated tests. The framework introduces an *adaptive focal context* mechanism that extracts only the relevant method, class properties, and dependencies via AST parsing, rather than feeding the entire source file to the model. Generated tests pass through a Generation-Validation-Repair cycle that includes syntactic, compilation, and runtime checks, with both rule-based and LLM-based repair strategies. In evaluation, ChatUniTest achieved 59.6% line coverage, outperforming TestSpark (42.1%) and EvoSuite (38.2%). This paper is directly relevant to our project since our workflow similarly relies on iterative prompt construction and post-generation validation to obtain compilable and correct unit tests from LLMs.

1.2.2 Mutation-Guided LLM-Based Test Generation at Meta

Foster et al. [2] describe Meta's *Automated Compliance Hardening (ACH)* system, which combines mutation testing with LLM-based test generation. Rather than prompting an LLM to "write tests" generically, ACH first generates targeted mutants—simulated faults specific to a concern such as privacy—and then instructs the LLM to produce tests that kill these mutants. Deployed on 10,795 Kotlin classes across seven Meta platforms, ACH generated 571 hardening test cases with a 73% developer acceptance rate. A notable finding is that 49% of the valuable tests did not increase line coverage, demonstrating that coverage alone is an insufficient quality metric. This insight directly informs our manual assessment phase, where we complement coverage analysis with equivalence class partitioning and boundary testing.

1.2.3 Adaptive Testing for LLM-Based Applications

Yoon et al. [3] address the challenge of efficiently testing LLM-powered applications by adapting *Adaptive Random Testing (ART)* to text-based prompt templates. Their approach iteratively selects the most diverse candidate input—measured via string distance metrics such as Levenshtein and Jaccard—and dynamically adjusts selection based on prior pass/fail outcomes. The technique discovers failures significantly faster than random sampling while requiring fewer costly API calls. For our project, this work provides a theoretical foundation for the input diversity strategies we apply when designing equivalence classes and selecting varied test inputs from the HumanEval dataset.

1.2.4 LLM-Based Code Generation for Golang Compiler Testing

Gu et al. [4] investigate using LLMs as intelligent fuzzers for compiler testing. By fine-tuning an LLM on a carefully filtered Go codebase and guiding generation with a coverage-based seed scheduling strategy, the authors achieved 3.38% compiler code coverage—nearly eight times higher than standard Go fuzzing (0.44%). The generated code exhibited only 2.79% syntax errors and effectively zero undefined behaviors. Although our project targets Java unit tests rather than compiler fuzzing, this paper demonstrates how constrained, seed-guided LLM generation can produce high-quality code artifacts, reinforcing the value of prompt engineering and structured generation strategies in our own pipeline.

1.2.5 LLM-Based Test-Driven Interactive Code Generation

Fakhoury et al. [5] propose *TiCoder*, an interactive workflow where the LLM first generates unit tests from a natural language prompt to formalize user intent, and then produces the implementation code constrained by those tests. In a user study with 15 programmers, participants using TiCoder were significantly more likely to correctly evaluate AI-generated code and reported lower cognitive load. At scale, the approach improved pass@1 accuracy by approximately 46% within five interaction rounds. This paper is highly relevant to our project: it validates the test-driven philosophy we adopt in Phase 1, where LLM-generated tests serve as the specification against which code correctness is evaluated, and it motivates the iterative refinement approach we use in both our agentic and semi-agentic workflows.

2 LLM Selection and Methodology

2.1 Selected LLMs

For this project, we selected two distinct Large Language Models to generate solutions and test cases for the HumanEval dataset prompts. The selection was based on their state-of-the-art performance in code generation and reasoning tasks.

Table 1: Comparison of Selected LLMs

Criterion	**LLM 1**	**LLM 2**
Model Name	Gemini Pro	Claude Opus 4.6
Access Method	Web Interface (Semi-Agentic)	Web Interface (Semi-Agentic)
Primary Role	Code and test generation	Code and test generation

2.2 Semi-Agentic Approach Justification

Initially, we attempted to implement a fully automated agentic approach. We designed Python scripts to automate the prompting, code extraction, and test generation pipeline directly via the models' APIs. However, we encountered significant challenges regarding API rate limits and token exhaustion. The continuous stream of requests required for 30 HumanEval tasks, including generation, validation, and potential repair cycles, quickly depleted our free token quotas and incurred prohibitive costs.

To bypass these API constraints and complete the project successfully, we pivoted to a *semi-agentic approach*. In this workflow, we manually managed the interaction with the LLMs through their respective web interfaces. We crafted the prompts locally, submitted them to the web UI, and manually extracted the generated code and tests to integrate them into our local Maven project.

2.3 Advantages and Limitations

The semi-agentic approach presented distinct advantages and limitations compared to a fully automated pipeline.

Advantages:

- *Cost Control:* By utilizing the web interfaces, we completely circumvented the API costs and token limits that stalled our initial automated approach.
- *Enhanced Control and Observation:* Manually handling the prompts allowed us to closely monitor the models' reasoning processes in real-time, making it easier to adjust our prompt engineering strategies on the fly when models hallucinated or failed to follow instructions.

Limitations:

- *Increased Manual Effort:* The most significant drawback was the substantial manual labor required to copy, paste, format, and organize the code for 30 distinct tasks across two models.
- *Reduced Scalability:* While effective for a subset of 30 prompts, this manual workflow is not scalable for evaluating the entire HumanEval dataset (164 tasks) or for integration into continuous integration/continuous deployment (CI/CD) pipelines.

3 Phase 1: Unit Testing and Manual Assessment

3.1 Prompt Selection Criteria

3.2 Difficulty Categorization

Table 2: HumanEval Prompt Categorization by Difficulty

Difficulty	Task IDs	Description / Rationale
Easy		
Moderate		
Hard		

3.3 Base Testing Phase

3.4 Manual Assessment Results

Table 3: Equivalence Class Partitioning – Task #X

Class ID	Type	Description	Expected Output
EC1	Valid		
EC2	Valid		
EC3	Invalid		
EC4	Invalid		
Boundary Conditions			
BC1			
BC2			

3.5 Test Smell and Branch Coverage Analysis

Table 4: Test Smell Analysis Summary

Task	Smell Type	Description	Action Taken

Table 5: Branch Coverage Summary

Task	Base Coverage (%)	Improved Coverage (%)	Improvement

3.6 Coverage Discussion and Input Mutations

4 Phase 2: Integration Testing

4.1 BookScan Class Design

Figure 1: BookScan Class UML Diagram

4.2 Task Integration Details

4.2.1 Task #18 – Substring

4.2.2 Task #23 – String Length

4.2.3 Task #27 – Upper-Lower Case

4.3 Integration Tests

Table 6: Integration Test Results Summary

Test Scenario	LLM 1	LLM 2	Notes
	Pass/Fail	Pass/Fail	

4.4 Complex Integration Evaluation

4.5 Prompt Engineering: Unmodified vs. Edited Prompts

Table 7: Prompt Engineering Comparison

Metric	Unmodified	Edited	Observation
Code Correctness			
Test Pass Rate			
Coverage			
Code Quality			

5 Results, Analysis, and Refactoring

5.1 Test Case Results

Table 8: Overall Test Results by LLM

Metric	LLM 1 (Unit)	LLM 1 (Integ.)	LLM 2 (Unit)	LLM 2 (Integ.)
Total Tests				
Passed				
Failed				
Pass Rate (%)				
Code Correctness				

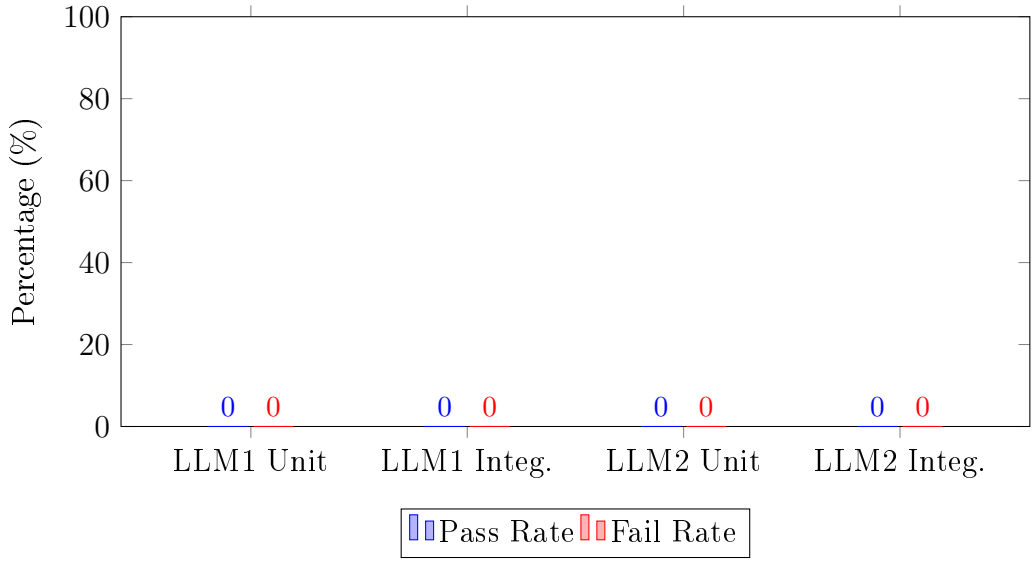

Figure 2: Test Pass/Fail Rates by LLM and Test Type

5.2 Statistical Evaluation

5.3 LLM Comparison

Table 9: Per-Method Comparison Between LLMs

Task	LLM 1 Coverage (%)	LLM 1 Tests	LLM 2 Coverage (%)	LLM 2 Tests

5.4 Refactoring Process

5.5 Agent-Generated Code Errors

Table 10: Common Errors in Agent-Generated Code

Task	Error Type	Description	Resolution

6 Conclusion

7 Acknowledgments

7.1 GitHub Repository

The source code and all artifacts for this project are available at:

https://github.com/YOUR_REPO_URL_HERE

7.2 Work Distribution

Table 11: Work Distribution Among Team Members

Member	Student ID	Responsibilities
Ahmet Enes Çiğdem	150220079	
Ali Eren Çiftçi	150220022	
Taha Çali	150220050	

References

[1] Y. Chen, Z. Hu, C. Zhi, J. Han, S. Deng, and J. Yin, "ChatUniTest: A framework for LLM-based test generation," in *Companion Proc. 32nd ACM Int. Conf. Foundations of Software Engineering (FSE Companion '24)*, Porto de Galinhas, Brazil, Jul. 2024, pp. 572–576. https://doi.org/10.1145/3663529.3663801

[2] C. Foster, A. Gulati, M. Harman, I. Harper, K. Mao, J. Ritchey, H. Robert, and S. Sengupta, "Mutation-guided LLM-based test generation at Meta," in *33rd ACM Int. Conf. Foundations of Software Engineering (FSE Companion '25)*, Trondheim, Norway, Jun. 2025. https://doi.org/10.1145/3696630.3728544

[3] J. Yoon, R. Feldt, and S. Yoo, "Adaptive testing for LLM-based applications: A diversity-based approach," in *Proc. IEEE/ACM Int. Conf. Software Engineering (ICSE '25)*, 2025.

[4] Q. Gu *et al.*, "LLM-based code generation method for Golang compiler testing," in *Proc. 31st ACM Joint European Software Engineering Conf. and Symp. on the Foundations of Software Engineering (ESEC/FSE '23)*, San Francisco, CA, USA, Dec. 2023.

[5] S. Fakhoury, A. Naik, G. Sakkas, S. Chakraborty, and S. K. Lahiri, "LLM-based test-driven interactive code generation: User study and empirical evaluation," *IEEE Trans. Softw. Eng.*, vol. 50, no. 9, pp. 2254–2267, Sep. 2024. https://doi.org/10.1109/TSE.2024.3428972

[6] M. Chen *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.